

Лабораторная работа по теме №3 "Проигрывание звуковых файлов. Формат WAV. Библиотека XAudio2"

Цель работы: В лабораторной работе предлагается изучить WAV формат. В качестве инструмента при выполнении лабораторной работы можно использовать любую систему программирования, позволяющую считывать и выводить файлы. Дается шаблон программы на языке C++.

Теоретическое обоснование

Подсистема **XAudio2** — это современный низкоуровневый аудио-API, который обеспечивает маршрутизацию и смешивание звука с минимальными задержками, что идеально подходит для высокопроизводительных игровых приложений. Он работает как на программном уровне, так и может использовать аппаратное ускорение, если оно доступно. XAudio2 предоставляет следующие возможности:

- **Получение и обработка звуковых данных** от приложения.
- **Смешивание и обработка** большого количества аудиопотоков.
- **Применение эффектов** (Audio Processing Objects, APO) к звуку.
- **Вывод результирующего аудиосигнала** на конечное устройство.

Приложению не приходится вникать в детали программирования конкретного аудиоадаптера — это остается задачей нижележащих уровней операционной системы. Вместо этого приложению предоставляется гибкая и мощная модель обработки звука. С уровня приложения имеются следующие основные возможности:

1. **Смешивание сигналов.** XAudio2 позволяет приложению создавать множество источников звука (source voices), которые будут обработаны и воспроизведены одновременно. Количество таких источников ограничено только мощностью центрального процессора.

2. **Объемный звук.** Встроенная поддержка 3D-звука позволяет размещать монофонические или многоканальные источники звука в трехмерном пространстве. Для каждого источника, а также для самого слушателя, задаются координаты, ориентация и скорость. XAudio2 обрабатывает все источники и создает для слушателя объемную и реалистичную звуковую картину с учетом затухания, эффекта Доплера и других параметров.

3. **Звуковые буферы.** XAudio2 работает с буферами данных, которые приложение отправляет для воспроизведения. Приложение заполняет эти буферы PCM-данными из

WAV-файла или другого источника и передает их в *голос источника* (*source voice*). Можно воспроизводить как небольшие звуковые эффекты из статических буферов в памяти, так и длинные музыкальные треки с помощью потоковой передачи данных.

Введение в XAudio2

XAudio2 — это программный интерфейс (API), входящий в состав Windows SDK и являющийся преемником DirectSound. Эта библиотека была разработана для поддержки высокопроизводительных мультимедиа-приложений и игр.

Название DirectX, частью которого исторически был DirectSound, трактуется буквально — «прямой, непосредственный интерфейс X». Классические аудио-интерфейсы Windows были созданы во времена, когда приложениям требовалось в первую очередь проигрывать длительные непрерывные звукозаписи. С массовым переходом производителей игр на платформу Windows выяснилось, что классические интерфейсы слишком абстрактны и вносят заметные задержки, что неприемлемо для игр, где требуется мгновенная реакция и одновременное воспроизведение множества коротких звуков. **XAudio2** был разработан именно для того, чтобы предоставить разработчикам эффективный и гибкий низкоуровневый интерфейс для решения этих задач.

Современные компоненты для работы с мультимедиа в Windows включают:

1. **DirectX Graphics** (Direct3D 11/12): API для программирования всей 2D и 3D графики.
2. **XAudio2**: Низкоуровневый API для обработки и воспроизведения звука.
3. **DirectInput** / **XInput**: API для поддержки множества устройств ввода (клавиатуры, мыши, геймпады).
4. **Windows Networking** (WinSock и др.): API для поддержки сетевых игр и приложений.
5. **Media Foundation**: Высокоуровневый API для захвата и воспроизведения мультимедийных потоков, преемник DirectShow.

Назначение и структура XAudio2

Подсистема **XAudio2** обеспечивает приложениям высокопроизводительный доступ к средствам обработки и вывода звука. При этом приложение избавляется от необходимости работать напрямую с драйверами конкретных аудиоустройств.

XAudio2 построен по объектно-ориентированному принципу в соответствии с моделью COM (Component Object Model) и состоит из набора интерфейсов. Каждый интерфейс отвечает за объект определенного типа — главный объект XAudio2, голос источника, голос микширования и т.п. По сути, интерфейс представляет собой обычный набор управляющих функций (методов), организованных в COM-класс.

XAudio2 напрямую не поддерживает сжатые звуковые форматы (MP3, WMA и др.). Он работает с несжатыми данными в формате PCM (Pulse Code Modulation). PCM — это формат звуковых данных в "чистом" виде, где значения показывают амплитуду звука в каждый момент времени. Назначение XAudio2 — исключительно эффективный вывод и обработка звука, а операции по декодированию форматов остаются в ведении приложений, которые могут использовать для этого другие библиотеки, например, Media Foundation.

Основы воспроизведения в XAudio2

Для воспроизведения в XAudio2 используется так называемый *граф обработки звука*, состоящий из *голосов* (voices):

- **Голос источника (Source Voice):** Объект, который принимает и обрабатывает аудиоданные от приложения. Данные поступают в виде буферов с PCM-сэмплами. Вы можете создать столько голосов источников, сколько звуков хотите воспроизводить одновременно. Для каждого голоса можно индивидуально настраивать громкость, высоту тона и другие параметры.
- **Промежуточный голос (Submix Voice):** Необязательный объект, который принимает звук от одного или нескольких других голосов (источников или других промежуточных голосов), смешивает их и может применять к общему сигналу один или несколько эффектов (например, реверберацию). Это позволяет группировать звуки (например, все звуки выстрелов) и управлять ими как единым целым.
- **Мастер-голос (Mastering Voice):** Конечная точка графа. Этот объект представляет конкретное аудиоустройство вывода (например, колонки). Он принимает звук от голосов источников и/или промежуточных голосов, производит финальное микширование и отправляет результат на аппаратуру. Мастер-голос создается один раз для каждого аудиоустройства.

Связь между вышеперечисленными элементами представлена на рисунке. Голоса источников отправляют свои данные на промежуточные и/или мастер-голос.

Основы программирования XAudio2

Для того чтобы использовать функции XAudio2 в вашем проекте, необходимо совершить следующие шаги:

1. Включить заголовочный файл:

```
#include <xaudio2.h>
```

2. **Прилинковать библиотеку:** в свойствах проекта добавьте библиотеку **xaudio2.lib** или вставьте строку:

```
#pragma comment(lib, "xaudio2.lib")
```

3. Инициализировать COM: так как XAudio2 — это COM-компонент, перед его использованием необходимо инициализировать COM-библиотеку:

```
CoInitializeEx(nullptr, COINIT_MULTITHREADED);
```

Поскольку в XAudio2 нет встроенных функций для чтения WAV-файлов, вам придется реализовать эту логику самостоятельно или использовать готовый код из примеров Windows SDK. Стандартный подход — написать функцию, которая открывает WAV-файл, считывает его заголовок (структуру WAVEFORMATEX) и находит начало блока данных.

Рассмотрим основные шаги для воспроизведения звука:

1. Создание объекта XAudio2:

```
IXAudio2* pXAudio2 = nullptr;  
XAudio2Create(&pXAudio2, 0, XAUDIO2_DEFAULT_PROCESSOR);
```

Это главный объект, управляющий всеми остальными.

2. Создание мастер-голоса:

```
IXAudio2MasteringVoice* pMasterVoice = nullptr;  
pXAudio2->CreateMasteringVoice(&pMasterVoice);
```

Это "выход" на ваши колонки или наушники.

3. Чтение WAV-файла и загрузка данных:

- Прочитайте заголовок файла, чтобы получить структуру WAVEFORMATEX. Она описывает формат аудио (количество каналов, частоту дискретизации, битность).
- Загрузите сами аудиоданные (PCM-сэмплы) в буфер в памяти (std::vector<BYTE> или BYTE*).

4. Создание голоса источника:

```
IXAudio2SourceVoice* pSourceVoice = nullptr;  
// WAVEFORMATEX wfx = {0}; // Заполняется из WAV-файла  
pXAudio2->CreateSourceVoice(&pSourceVoice, &wfx);
```

Этот голос будет "проигрывать" наши данные. Формат голоса должен совпадать с форматом данных из файла.

5. Передача данных голосу источника:

```
XAUDIO2_BUFFER buffer = {0};  
// buffer.pAudioData = ...; // Указатель на данные из шага 3  
// buffer.AudioBytes = ...; // Размер данных в байтах  
buffer.Flags = XAUDIO2_END_OF_STREAM; // Сообщает, что это последний буфер  
pSourceVoice->SubmitSourceBuffer(&buffer);
```

6. Запуск воспроизведения:

```
pSourceVoice->Start(0);
```

6. **Ожидание и очистка:** Программа должна работать, пока звук проигрывается. По завершении необходимо остановить голос и освободить все ресурсы в обратном порядке их создания.

Формат WAV файла

WAV файл — это звуковой файл формата RIFF. WAV-файл состоит из трех блоков – двух заголовочных и одного блока звуковых данных. Первый блок имеет идентификатор "RIFF", за которым в 4-х байтах следует размер файла (без учета первых 8 байтов). В следующих 4-х байтах стоит идентификатор "WAVE", указывающий тип RIFF-файла. (первый заголовок 12 байт)

Следующий заголовочный блок содержит байты в следующем порядке:

- 4 байта – идентификатор "fmt " (с пробелом)
- 4 байта – размер оставшейся части этого блока (обычно 16 для PCM)
- 2 байта – код сжатия (1 для PCM)
- 2 байта – число каналов (1 – моно, 2 – стерео)
- 4 байта – частота дискретизации (например, 44100)
- 4 байта – байт-рейт ($\text{Частота} * \text{Каналы} * \text{БитыНаСэмпл} / 8$)
- 2 байта – выравнивание блока ($\text{Каналы} * \text{БитыНаСэмпл} / 8$)
- 2 байта – число бит на сэмпл (например, 16)

Последний блок – данные отсчетов – начинается идентификатором "data", за которым расположены 4 байта – размер блока данных, а затем сами данные. В случае стереоотсчета данные для обоих каналов следуют друг за другом: сначала сэмпл для левого канала, затем сэмпл для правого, и т.д.

Файл может дополнительно содержать фрагменты других типов, поэтому не следует думать, что заголовок wav-файла имеет фиксированный формат. Например, в файле может присутствовать фрагмент "LIST" с подфрагментом "INFO", содержащий информацию о правах копирования и другую дополнительную информацию.

INFO - специальный ListType для хранения информации о содержимом файла. INFO не влияет на то, как программы работают с файлом, эта информация (большей частью) показывается пользователю.

Список chunk'ов для INFO:

- **IARL** (Archival Location) - место архивного хранения документа
- **IART** (Artist) - список авторов произведения (стандартный тэг, отображается практически во всех плеерах).
- **ICMS** (Commissioned) - список лиц, предоставивших содержимое файла.
- **ICMT** (Comments) - комментарий (отображается практически во всех плеерах).

- **ICOP** (Copyright) - информация об авторских правах.
- **ICRD** (Creation date) - Дата создания оригинального произведения. Формат YYYY-MM-DD.
- **ICRP** (Cropped) - данные об обрезке произведения.
- **IDIM** (Dimensions) - Физические размеры оригинала.
- **IENG** (Engineer) - фамилии, создававших файл.
- **IGNR** (Genre) - жанр (частично поддерживается).

Задание на работу

1. Изучите функции **XAudio2** и технологию их применения в среде Microsoft Visual Studio. В первую очередь используйте официальную документацию (MSDN) и Windows SDK.
2. Создайте консольное приложение для проигрывания wav-файла на основе примера из методических указаний.
3. Добавьте в программу новые функции оригинального проигрывания, например, эхо, изменение высоты тона, одновременное проигрывание нескольких файлов или др.
4. Измените программу так, чтобы она изменяла заголовок WAV-файла (вносила ваше имя в заголовок) и сохраняла файл с новыми эффектами на диск.

Проигрыватель WAV файлов

Теперь на основе этих принципов несложно написать проигрыватель Wav файлов:

```
#include <windows.h>
#include <xaudio2.h>
#include <iostream>
#include <string>
#include <vector>
#include <fstream>
#include <conio.h>

// Подключение библиотеки XAudio2
#pragma comment(lib, "xaudio2.lib")

int main() {
    // Шаг 0: Настройка потоков ввода/вывода и глобальных переменных
    HRESULT hr = S_OK;
    IXAudio2* pXAudio2 = nullptr;
    IXAudio2MasteringVoice* pMasterVoice = nullptr;
    IXAudio2SourceVoice* pSourceVoice = nullptr;
    // Формат аудио считывается из WAV-данных и подстраивается под PCM
    WAVEFORMATEX wfx = {};
    std::vector<uint8_t> wavData;
    struct WAVHeader {
        uint32_t riff;
        uint32_t size0;
        uint32_t wave;
        uint32_t fmt;
```

```

    uint32_t fmtSize;
    uint16_t audioFormat;
    uint16_t channels;
    uint32_t sampleRate;
    uint32_t avgBytesPerSec;
    uint16_t blockAlign;
    uint16_t bitsPerSample;
    uint32_t data;
    uint32_t dataSize;
} header = {};

// Шаг 1: Ввод пути к WAV-файлу во время выполнения
std::string filename;
std::cout << "Enter the full path to input WAV file: ";
std::getline(std::cin, filename);
if (filename.empty()) {
    std::cerr << "There is no such file. Exiting.\n";
    return -1;
}

// Шаг 2: Инициализация COM-подсистемы и XAudio2
hr = CoInitializeEx(nullptr, COINIT_MULTITHREADED);
if (FAILED(hr)) {
    std::cerr << "Can't initialize COM: " << std::hex << hr << "\n";
    return -1;
}

hr = ::XAudio2Create(&pXAudio2, 0);
if (FAILED(hr)) {
    std::cerr << "Can't create XAudio2: " << std::hex << hr << "\n";
    CoUninitialize();
    return -1;
}

// Шаг 3: Создание мастер-голоса (Mastering Voice)
hr = pXAudio2->CreateMasteringVoice(&pMasterVoice);
if (FAILED(hr)) {
    std::cerr << "Can't create Mastering Voice: " << std::hex << hr << "\n";
    pXAudio2->Release();
    CoUninitialize();
    return -1;
}

// Шаг 4: Встроенная загрузка WAV-файла внутри main
// Читаем простой RIFF/WAV заголовок и данные без обработки
{
    std::ifstream file(filename, std::ios::binary);
    if (!file) {
        std::cerr << "Can't open WAV file: " << filename << "\n";
        pMasterVoice->DestroyVoice();
        pXAudio2->Release();
        CoUninitialize();
        return -1;
    }

    // 1) RIFF заголовок
    file.read(reinterpret_cast<char*>(&header), sizeof(WAVHeader));
    if (!file) {
        std::cerr << "Can't read WAV-header.\n";
        file.close();
        pMasterVoice->DestroyVoice();
        pXAudio2->Release();
        CoUninitialize();
        return -1;
    }
}

```

```

// 2) Чтение "данных" чанка
wavData.resize(header.dataSize);
file.read(reinterpret_cast<char*>(wavData.data()), header.dataSize);
if (!file) {
    std::cerr << "Can't read WAV-data.\n";
    file.close();
    pMasterVoice->DestroyVoice();
    pXAudio2->Release();
    CoUninitialize();
    return -1;
}
file.close();

// Заполняем параметры WAV-файла в WAVEFORMATEX
wfx.wFormatTag = 1; // WAVE_FORMAT_PCM
header.fmt = header.fmt; //
wfx.nChannels = static_cast<WORD>(header.channels);
wfx.nSamplesPerSec = header.sampleRate;
wfx.wBitsPerSample = static_cast<WORD>(header.bitsPerSample);
wfx.nBlockAlign = (wfx.nChannels * wfx.wBitsPerSample) / 8;
wfx.nAvgBytesPerSec = wfx.nSamplesPerSec * wfx.nBlockAlign;
wfx.cbSize = 0;
}

// Шаг 5: Создание Source Voice с указанием формата
hr = pXAudio2->CreateSourceVoice(&pSourceVoice, &wfx);
if (FAILED(hr)) {
    std::cerr << "Can't create Source Voice: " << std::hex << hr << "\n";
    pMasterVoice->DestroyVoice();
    pXAudio2->Release();
    CoUninitialize();
    return -1;
}

// Шаг 6: Подготовка XAUDIO2_BUFFER и передача данных
XAUDIO2_BUFFER buffer = {};
buffer.AudioBytes = static_cast<UINT32>(wavData.size());
buffer.pAudioData = wavData.data();
buffer.Flags = XAUDIO2_END_OF_STREAM;

hr = pSourceVoice->SubmitSourceBuffer(&buffer);
if (FAILED(hr)) {
    std::cerr << "Can't send buffer in Source Voice: " << std::hex << hr << "\n";
    pSourceVoice->DestroyVoice();
    pMasterVoice->DestroyVoice();
    pXAudio2->Release();
    CoUninitialize();
    return -1;
}

// Шаг 7: Запуск воспроизведения
hr = pSourceVoice->Start(0);
if (FAILED(hr)) {
    std::cerr << "Can't start Source Voice: " << std::hex << hr << "\n";
    pSourceVoice->DestroyVoice();
    pMasterVoice->DestroyVoice();
    pXAudio2->Release();
    CoUninitialize();
    return -1;
}

// Шаг 8: Ожидание завершения воспроизведения
XAUDIO2_VOICE_STATE state;
do {

```



```

        pSourceVoice->GetState(&state);
        Sleep(100);
    } while (state BuffersQueued > 0);

    // Шаг 9: Очистка ресурсов
    pSourceVoice->DestroyVoice();
    pMasterVoice->DestroyVoice();
    pXAudio2->Release();
    CoUninitialize();

    // Готово
    std::cout << "Playback complete.\n";
    return 0;
}

```

Содержание отчета

1. Цель работы.
2. Задание на работу.
3. Исходный код и скриншот вывода для четвёртого задания с указанием используемых эффектов.
4. Выводы по работе.